GOOGLE GEMMA 4 DEVELOPER AGENT COMPETITION · PAPER TRACK (TASKS, BENCHMARKS & NEW RESOURCE)

qb-cubie in a Test-Guided Coding Agent: Methodology, Development, and Exploratory Pilot Findings

A Zero-Leakage A/B Evaluation of Example-Based Testing (qb-cubie 0.0.3) vs. pytest on Everyday Consumer Hardware Across Single-Agent (v1–v5) and Multi-Agent (v6–v9) Architectures

Author & Project Lead: Dominick Wirzba · Paper Synthesis Origin: Gemini-App (Gemini-4-Argon) · Telemetry & Audit Analyses: Antigravity (Gemini-3.8-Flash) & Codex (GPT-6.1-Sol)

Data Cutoff: November 12, 2026 (v1–v5 Cutoff: Nov 11, 2026; v6–v9 Cutoff: Nov 12, 2026) · Artifacts: Published on PyPI (qb-cubie 0.0.3) and GitHub (MIT License).

ABSTRACT — Deploying open-weights coding agents on everyday consumer hardware requires evaluation harnesses that separate genuine code repair from test manipulation. We present qb-cubie (v0.0.3), an open-source Python quality assurance framework providing declarative Example-Based Testing (EBT), and evaluate whether providing qb-cubie and its structured guide improves the resolution rate of a local gemma4:12b agent (11.9B, GGUF Q4_K_M via Ollama 0.40.2) on real-world SWE-bench Python tasks compared to standard pytest. Both arms operate under identical budgets and repository snapshots, while an independent, Cubie-free container strips all agent test edits and evaluates only the production patch against private FAIL_TO_PASS and PASS_TO_PASS suites.

Across the single-agent series (v1–v5; 7 completed pilots, 42 runs on 3 tasks with Seed 42, backed by 5,563 verified SHA-256 hashes), private resolutions occurred in 3 of 42 runs (7.1%): twice with pytest (v2, v4) and once with qb-cubie (revised v3). However, EBT contracts were executed in only 2 of 21 Cubie runs (9.5%) and remained unused in 19 of 21 runs (90.5%). In Pilot v5 (think=true), both arms resolved 0/3 tasks: 11 of 67 model calls hit the 8,192-token cap emitting only thinking tokens without a JSON action, consuming 87.3% (90,112 of 103,235) of output tokens and 79.6% of agent runtime. A follow-up series across four serial multi-agent generations (v6–v9: Analysis, QA, Dev; 17 runs, 1,441 model calls, 8.55M input tokens) reduced infrastructural errors to 0 cursor errors, 0 tool errors, and 0 context overflows in v9. Yet removing v8's forced handoff while overloading the finish tool schema caused the model to invoke finish zero times (0 calls) across 1,184 steps (974 duplicates, 82.6%), locking execution in Analysis before the QA-scoped qb-cubie guide could be injected.

1. Introduction

Running autonomous software engineering agents on everyday consumer hardware using compact open-weights models such as gemma4:12b (11.9B parameters, 4-bit quantized Q4_K_M) exposes operational bottlenecks that are often masked by frontier cloud APIs or permissive evaluation harnesses. When an agent is tasked with reproducing a bug, writing a test, and repairing production code, failure to resolve an issue rarely stems from a single cause; instead, inference limits, pre-training biases, noisy repository baselines, and tool-interface friction interact as a compound bottleneck cascade.

To investigate how test methodology interacts with these constraints, we introduce qb-cubie (evaluated at version 0.0.3, commit 5e50d22f80db90c4e8ae6e3aeedab6acdb981230) and conduct a cryptographically frozen A/B study asking (Protocol, 2026; Single-Agent Report, 2026): Does access to qb-cubie and its specified guidance improve the independently verified resolution rate of a test-guided coding agent powered by Ollama and gemma4:12b under a fixed working budget compared to the exact same agent using standard pytest? Because the intervention pairs the library with its instructional prompt (Study Guides, 2026), the study measures the combined package offering rather than isolating library mechanics from prompt effects.

1.1 Declarative Contract Semantics of qb-cubie and Epistemological Boundaries

The open-source package qb-cubie unifies declarative Example-Based Testing (EBT), natural-language Review-Based Testing (RBT), execution arenas (TempFS, PyVenv), static analysis orchestrators (AstLinter, CstLinter), and a benchmark subsystem (qb_cubie.benchmarks / cubie-bench) with native SWE-bench support (Contract Semantics, 2026; Benchmark Guide, 2026; Linter Guide, 2026; RBT Usage Pattern, 2026). An EBT contract defines a finite set of concrete executions of a callable target via mutable WorldState dataclasses (fields: args, kwargs, arena, fs, returns, expect, raises, exception). Given input states P in pre, permitted patterns Q+ (post.expect=True), and forbidden patterns Q− (post.expect=False), where match(p, q) checks return values, expected exceptions via ExceptionExpectation, and filesystem diffs in fs, the finite evaluation rule is:

```
step_ok(p) = (Q+ is empty OR at least one q in Q+ matches) AND (no q in Q− matches)
run_ok     = all step_ok(p) hold;   evaluation = run_ok if EBT.expect=True else NOT
run_ok
```

While the structure “pre-state – operation – post-state” is inspired by a Hoare triple {P} S {Q}, WorldState is a concrete data pattern rather than a formal state predicate (Quality Assurance Guide, 2026). Passing an EBT contract demonstrates observed conformance over finite inputs rather than a formal proof. Because all inputs in pre are checked against the shared set post, correlated input/output pairs must use separate EBT contracts (e.g., via pytest.mark.parametrize), and WorldState(returns=None) explicitly requires None whereas qb_cubie.UNSPECIFIED disables return checking (EBT Guide, 2026; EBT Usage Pattern, 2026). In our study, an EBT contract counts as utilized only if executed via .run() and verified via assert result.evaluation is True (Single-Agent Report, 2026).

An internal audit of the repository (Technical Audit, 2026), where 2,569 tests pass in 550.50s (Python 3.13.12), documented three core edge cases that did not affect the agent runs: (B1, P1) PyVenv stores IPC files (ipc_in.pkl, ipc_out.pkl, _cubie_phantom.py) inside the workspace (Arena Management, 2026); (B2, P1) PytestReporter uses assert statements ignored under python -O; and (B3, P2) CstLinter.check() returns [] on missing paths. Following an early audit warning against circular validation, all 698+ verification nodes for the study controllers were authored with standard pytest assertions rather than EBT contracts (Technical Audit, 2026).

2. Related Works

Repository-level coding benchmarks are anchored by SWE-bench, SWE-bench Lite, and SWE-bench Verified, which evaluate whether patches resolve real GitHub issues against Fail-to-Pass and Pass-to-Pass test suites (Benchmark Guide, 2026). Agentic harnesses such as SWE-agent, AutoCodeRover, and Agentless demonstrate that structured navigation, search, and localized editing tools strongly influence resolution rates, though most published evaluations rely on large frontier models. Our work complements these studies in three ways: (1) we introduce qb-cubie as a declarative EBT alternative to procedural pytest assertions; (2) we enforce a strict zero-leakage evaluation boundary that allows agents to freely edit visible tests during test-driven development while stripping all test edits prior to private container evaluation (Validation Report, 2026); and (3) we provide a granular failure taxonomy of a 4-bit quantized 12B open-weights model across nine single- and multi-agent architectures (Antigravity Analysis, 2026; Codex Analysis, 2026).

3. Methodology, SWE-bench Dataset, and Zero-Leakage Harness

The study task pool comprises 129 real-world GitHub pull requests and issues structured in the standard SWE-bench format (instance_id, repo, base_commit, patch, test_patch, problem_statement, hints_text) across four Python repositories: 67 from FastAPI (fastapi/fastapi), 48 from Rich (Textualize/rich), 13 from Requests (psf/requests), and 1 from HTTPX (encode/httpx) (Data Documentation, 2026; Submission Audit, 2026). In addition, cubie includes a native SWE-bench benchmark module under src/qb_cubie/benchmarks/swebench/ supporting official Hugging Face datasets (Benchmark Guide, 2026). A manual audit of 39 tasks (Candidate Audits, 2026) identified 26 qualified candidates, 9 conditional cases, 3 deferred cases, and 1 excluded case (with 90 tasks unaudited). Three tasks were frozen with Seed 42 for the iterative pilot series (Single-Agent Report, 2026):

Task ID	Repository & Category	Required Modification	FAIL_TO_PASS	PASS_TO_PASS
fastapi_9425	FastAPI · API/Schema	Treat stringified return annotation "None" in get_typed_return_annotation as None so HTTP 204 bodiless responses succeed.	1	8
requests_6589	Requests · Encoding	Use UTF-8 encoded byte length instead of character length for strings in super_len (Content-Length calculation).	1	11
rich_3278	Rich · ANSI Parsing	Strip private ANSI escape sequences from ESC 0 to ESC ? so console output is not corrupted.	16	7

Before each run, snapshots are sanitized of git history and reference patches. Both conditions (cubie and pytest) run serially from identical containers with alternating starting arms. During execution, agents may edit both production files and visible tests. Afterward, a fresh, Cubie-free evaluator container strips all test and runner edits, applies only the production patch, and runs the frozen private FAIL_TO_PASS and PASS_TO_PASS suites (Single-Agent Report, 2026). A task is resolved if and only if all private tests pass, regardless of whether the agent called submit.

Repository Autonomy and Cryptographic Integrity: The cubie package (pyproject.toml) has no dependency on external harnesses such as swegemma (Submission Audit, 2026). In early single-agent runs (v1–v5), a local shell wrapper (./study) invoked a neighboring virtual environment (../devagent-comp/.venv/bin/python) containing swegemma 0.2.11 for container bridging; from v6 onward (team_workspace.py, team_v9.py), workspace management runs purely via native git subprocesses and standard Docker containers (Submission Audit, 2026). Across all eight v1–v5 archives under runs/, 5,563 SHA-256 hash references across input manifests, preflight files, and run manifests were audited with 100% agreement (Single-Agent Report, 2026). In Pilot v5 (20261111-200641), gemma4:12b (digest 4eb23ef187e2c5462566d6a1d3bbbc2f1346d0b4327cbb66d58fffbcc9b2b05c) ran on Ollama 0.40.2 (Python 3.13.18, pytest 9.2.0, qb-cubie 0.0.3) with Seed 42, temperature 0.2, think=true, num_ctx=32768, num_predict=8192, 900 s agent limit (90 s command / 120 s verification), and 80,000 configured context bytes (40,090 effective bytes).

4. Single-Agent Experiments (Generations v1–v5)

The single-agent core evolved across five frozen designs (Agent Design v1–v5, 2026): early unversioned (basic tools, 600 s / 24 calls), v2 (working ledger, anchored edits, output repair), v3 (action-coupled notes, baseline counter-checks, EBT tracking; expanded in revised v3 to 900 s, unlimited calls, and 32k/8k tokens), v4 (runner-bound file hashes, oracle checks), and v5 (mandatory analysis checkpoints plus think=true). Table 2 summarizes all seven completed pilots (42 runs) and one prepared-only archive (180104; a separate 60 s smoke check in local/live-pilot-validation is excluded) (Single-Agent Report, 2026):

Archive ID	Core & Thinking	Budget (Time / Calls · Ctx / Out)	pytest Resolved	Cubie Resolved
132401 & 160158	early · think=false (2 pilots)	600 s / 24 · 16,384 / 4,096	0/3 & 0/3	0/3 & 0/3
163110	v2 · think=false	600 s / 24 · 16,384 / 4,096	1 / 3	0 / 3
171515 & 180104	v3 · think=false (180104 prep. only)	600 s / 24 · 16,384 / 4,096	0/3 & —	0/3 & —
180702	v3 rev. · think=false (1 unavail.)	900 s / unlim. · 32,768 / 8,192	0 / 3	1 / 3
192412	v4 · think=false	900 s / unlim. · 32,768 / 8,192	1 / 3	0 / 3
200641	v5 · think=true	900 s / unlim. · 32,768 / 8,192	0 / 3	0 / 3

- **Verified Resolutions & Exit-Code-4 Sensitivity:** Only 3 of 42 runs (7.1%) achieved private resolution: pytest on requests_6589 in v2 (163110); Cubie on fastapi_9425 in revised v3 (180702, run 0001, executing 6 EBT contracts with 5 passing); and pytest on fastapi_9425 in v4 (192412, where the production patch passed all 9 private tests [−33.3 pp, CI [−100; 0]], though the agent overwrote a 1,114-line visible test file with 45 broken lines and stalled) (Pilot Analyses, 2026). In 180702 (run 0004, Requests), the Cubie agent ran 3 EBT contracts but introduced an indentation error into production code, causing pytest exit code 4 during private collection. Treating run 0004 as unavailable yields +50.0 pp (CI [0; +100]) on 2 complete pairs; counting this agent-caused syntax error as a failure yields +33.3 pp across 3 complete pairs (Single-Agent Report, 2026).
- **90.5% EBT Non-Utilization & Baseline Pollution:** Across all 21 Cubie runs in v1–v5, EBT contracts were executed in only 2 runs (9.5%, both in 180702) and never executed in 19 runs (90.5%) (Submission Audit, 2026). Moreover, visible suites contained heavy pre-existing failures before agent edits (Technical Analysis, 2026): FastAPI had 8 collection errors (missing pydantic_settings); Requests had 577 passed, 5 failed, 15 skipped; and Rich had 746 passed, 65 failed, 24 skipped.

4.1 Deep Dive into Pilot v5 (Archive 20261111-200641)

Run	Task & Arm	Agent Time	Model / Tool Calls	Termination & Final Patch
0001 / 0002	FastAPI · Cubie / pytest	727.4 s / 681.4 s	8/6 & 6/4	Both invalid_output · both empty patches
0003 / 0004	Requests · pytest / Cubie	829.3 s / 900.1 s	17/15 & 19/17	invalid_output & agent_timeout · test edits only
0005 / 0006	Rich · Cubie / pytest	653.5 s / 684.7 s	8/6 & 9/7	Both invalid_output · both empty patches

In Pilot v5 (0/3 vs. 0/3; degenerate CI [0; 0]; 4,476.4 s total agent time [74.6 min], 80.3 min study wall time; median 684.7 s pytest vs. 727.4 s Cubie; 67 model calls [32 pytest, 35 Cubie], 55 tool calls [26 pytest, 29 Cubie], 1 vs. 4 tool errors, 0 EBT executions), documented tokens totaled 506,573 (252,363 in pytest [198,821 in / 53,542 out] plus 254,210 partial in Cubie due to the run 0004 timeout; 403,338 input and 103,235 output overall) (Single-Agent Report, 2026). Crucially, 11 of 67 model calls hit done_reason=length at 8,192 tokens with empty message.content and only thinking output, consuming 90,112 output tokens—87.3% of all known output tokens and 79.6% of agent runtime (Codex Analysis, 2026). On requests_6589, pytest (run 0003) wrote valid UTF-8 tests (super_len("€") == 3, super_len("Test€") == 7) that failed with 1 and 5, but its production edit failed with edit_conflict/anchor_missing; Cubie (run 0004) corrupted 🚀 into two UTF-16 surrogates ('\ud83d\ude80', length 2) in a plain assert without EBT, failing with 2 == 4 before timing out (Single-Agent Report, 2026).

5. Root-Cause Taxonomy and Factorial Program (Antigravity & Codex)

Independent post-hoc analyses by Antigravity (Antigravity Analysis, 2026) and Codex (Codex Analysis, 2026) formalize why the 7.1% resolution rate and 90.5% EBT non-utilization represent an upstream floor effect:

- **H1 (Reasoning Token Starvation):** Unbounded thinking in shared 8,192-token budgets displaces JSON actions (11/67 calls = 16.4% of calls, 87.3% of output tokens). Prediction: Capping reasoning at 2,048 tokens or raising num_predict to ≥ 16,384 cuts truncation below 10%; Codex Study A tests this via a 2×2 (v4/v5 × think=false/true) design.
- **H2 (Pre-Training Prior Dominance):** Under cognitive load, 12B models default to pre-trained pytest assertions when unrestricted file writing is allowed (19/21 Cubie runs without EBT), particularly on stateful tasks vs. pure functions (FastAPI in 180702). Prediction: Dedicated contract tools raise EBT execution above 75%.
- **H3 (Baseline Pollution & Edit Friction):** The 65 pre-existing failures in Rich act as a noise attractor (0/14 solved). Prediction: Scoping run_tests to target paths (Codex Study D) and replacing text anchors with opaque file handles (Codex Study B) halves stalling and edit conflicts.
- **H4 (Model Capability Threshold & Human-in-the-Loop Value):** Evaluating a 30B+ model under identical freezes (Codex Study C) is predicted to lift resolution above 30% and meet the campaign qualification gate (≥ 2 independent task groups resolved per arm with red–green verification). In human-in-the-loop usage on internal projects, the Project Lead observes that EBT contracts substantially improve test readability and oracle review speed (Single-Agent Report, 2026).

6. Multi-Agent Follow-Up Study (Generations v6–v9)

To test whether role specialization overcomes single-agent overload, a follow-up study on November 12 evaluated four serial multi-agent generations (v6–v9: Analysis, QA, and Dev sharing the workspace under gemma4:12b, Seed 42, think=false, temperature=0.15, num_ctx=32768, num_predict=8192, and up to 1,800 s budget) (Multi-Agent Designs v6–v9, 2026; Multi-Agent Synthesis v6–v9, 2026):

Generation & Archive	Exec. / Plan	Model / Tool Calls	Tokens (In / Out)	Reached Roles & Termination Cause
v6 (20261112-002038)	6 / 6	66 / 65	422,512 / 16,749	Analysis (100%) · finished: 6× stagnation stop (0/6 resolved)
v7-1 (20261112-013246)	3 / 6	46 / 45	316,839 / 5,649	Analysis (18), QA (27) · failed: post-freeze ValueError (0/3 resolved)
v7-2 (20261112-015457)	1 / 6	81 / 80	638,637 / 5,705	Analysis (100%) · interrupted: 32× find_paths loop (0/1 resolved)
v8 (20261112-030527)	1 / 6	64 / 63	195,452 / 2,302	Analysis (61), QA (2) · interrupted: 30+ cursor_error loop (0/1 resolved)
v9 (20261112-035522)	6 / 6	1,184 / 1,179	6,980,998 / 44,845	Analysis (100%) · finished: 6× budget_exhausted, 974 dup. (0/6 resolved)
Total (v6–v9)	17 / 30	1,441 / 1,432	8,554,438 / 75,250	Analysis dominant · 0 / 17 privately resolved (0.0%)

6.2 Error Domain Migration Across the Four Multi-Agent Generations (v6–v9)

Across all 1,432 tool actions in v6–v9, 90.7% were search_content (795, 55.5%) and read_file (504, 35.2%), alongside 72 read_evidence (5.0%), 52 find_paths (3.6%), 2 ask_analysis (0.1%), 3 finish_orientation (0.2%), 3 create_case (0.2%), 1 collect_tests (0.1%), and 0 finish calls (0.0%) (Multi-Agent Synthesis v6–v9, 2026):

1. **v6 (Archive 20261112-002038) — Two-Step Handoff Barrier:** Setting think=false restored JSON stability (65/66 valid calls; one 8,192-token regex loop in Rich repaired). Yet all 6 runs ended via stagnation stops (45–531 s) in orient without reaching QA or Dev; searching requests/utils.py instead of src/requests/utils.py returned empty hits, and the model never initiated publish_context followed by handoff, confirming Counter-Hypothesis V6-H8 (v6 Pilot Analyses, 2026).
2. **v7 (Archives 20261112-013246 & 20261112-015457) — Context Pollution & Tool-Removal Loops:** v7-1 reached QA (27 calls, 3 cases, 1 collection) before aborting on a post-freeze edit check. In v7-2, raw JSON ledgers and Cubie tutorials in the Analysis prompt truncated 55 tool turns; when recovery removed search_content, the model called the filename glob tool find_paths with "def get_route_info" 32 times in a row (Multi-Agent Synthesis v6–v9, 2026).
3. **v8 (Archive 20261112-030527) — Forced Handoff vs. Amnesia & Cursor Traps:** A deterministic controller rule (repeat_count >= 3 activating QA) successfully triggered QA, which sent two ask_analysis queries (job-12, job-20). Analysis found the exact target in fastapi/dependencies/utils.py (lines 253–280), but because it did not call update_context, the read files were lost (materials: [] in job-20), while passing "cursor": "0" triggered 30+ consecutive cursor_error rejections (Multi-Agent Synthesis v6–v9, 2026).
4. **v9 (Archive 20261112-035522) — Infrastructural Maturity vs. Controller-Prompt Lock-In:** v9 introduced transactional Zero-Admin Memory, cursor-free schemas, symmetric read/search rights, and clean Markdown prompts, while removing v8's forced handoff (Multi-Agent Designs v6–v9, 2026). Over 1,184 model calls and 1,179 tool steps (6.98M input tokens), infrastructural errors dropped to 0 cursor errors, 0 tool errors, and 0 context overflows, and Analysis located the target files within steps 0–13 on every task (v9 Pilot Analysis, 2026). However, removing the forced handoff while describing finish primarily with review parameters (verdict=verified/rework, which raised ValueError('Orientation cannot verify a repair') in orient at team_v9.py:381) caused the model to invoke finish zero times (0 calls) across 1,184 steps and fall into 974 duplicate actions (82.6%). Because team_v9.py:615 injects the framework guide only upon entering QA, the qb-cubie instructions were never presented to the model in any v9 run (Submission Audit, 2026; Revision-1 Grounding Audit, 2026).

7. Conclusion

Across nine agent generations (59 executed runs: 42 single-agent in v1–v5 and 17 multi-agent in v6–v9), we provide an open-source declarative testing framework (qb-cubie 0.0.3), a zero-leakage SWE-bench evaluation harness, and rigorous empirical evidence of the floor effects governing 12B models on everyday hardware. While flat single-agent architectures achieved three verified repairs—including a proof-of-concept resolution executing six EBT contracts in archive 180702—upstream bottlenecks (87.3% reasoning token starvation in v5, 90.5% EBT non-utilization from pytest priors, and phase-transition lock-in in v6–v9) prevented downstream test methodologies from being systematically exercised. Overcoming this floor requires decoupling reasoning budgets, enforcing dedicated contract-authoring tools, and pairing unambiguous phase transitions with higher-capacity models.

8. References and Frozen Study Artifacts

1. Single-Agent Report (2026): Wirzba, D. qb-cubie in einem testgeführten Programmieragenten (experiments/testguided-study/PAPER.md, Nov 11, 2026; v1–v5 through runs/20261111-200641).
2. Protocol (2026): Studienprotokoll der gepaarten Agentenstudie (experiments/testguided-study/PROTOKOLL.md).
3. Data Documentation (2026): SWE-bench Task Pool Documentation (experiments/testguided-study/DATEN.md).
4. Candidate Audits (2026): Einzelprüfungen & Katalog (experiments/testguided-study/KANDIDATEN.md, candidate-catalog.json).
5. Study Guides (2026): Agent Guides for qb-cubie and pytest (experiments/testguided-study/guides/cubie.md, pytest.md).
6. Validation Report (2026): Container Replay Validation (experiments/testguided-study/VALIDIERUNG.md).
7. Agent Design v1–v5 (2026): Single-Agent Designs (experiments/testguided-study/AGENTENDESIGN.md, V4.md, V5.md).
8. Pilot Analyses (2026): Analyses of Pilots v2–v4 (experiments/testguided-study/PILOTANALYSE-20261111.md, 192412.md).
9. Technical Analysis (2026): Visible Baseline Pollution Analysis (experiments/testguided-study/TECHNIKANALYSE-20261111-171515.md).
10. Antigravity Analysis (2026): Cascade Hypothesis System H1–H4 (experiments/testguided-study/HYPOTHESEN-URSACHENANALYSE-20261111.md, Gemini-3.8-Flash).
11. Codex Analysis (2026): Workflow Bottleneck & Factorial Program (experiments/testguided-study/URSACHENANALYSE-CODEX-20261111.md, GPT-6.1-Sol).
12. Benchmark Guide (2026): cubie-bench & Native SWE-bench Module (docs/GUIDE_Benchmarks.md, src/qb_cubie/benchmarks/swebench/).
13. Contract Semantics (2026): Verbindliche Vertragssemantik von Cubie 0.0.3 (docs/GUIDE_Contract-Semantics.md).
14. Quality Assurance Guide (2026): Qualitätssicherung mit Cubie (docs/GUIDE_Quality-Assurance.md).
15. EBT Guide (2026): Example-Based Testing (docs/GUIDE_Example-Based-Testing.md).
16. EBT Usage Pattern (2026): EBT-Nutzungsmuster (docs/GUIDE_EBT-Usage-Pattern.md).
17. Arena Management (2026): TempFS & PyVenv Arenas (docs/GUIDE_Arena-Management.md).
18. RBT Usage Pattern (2026): Review-Based Testing (docs/GUIDE_RBT-Usage-Pattern.md).
19. Linter Guide (2026): AstLinter & CstLinter (docs/GUIDE_Linter.md).
20. Technical Audit (2026): Kritisches Gutachten zu qb-cubie (./GUTACHTEN.md in project root, commit 5e50d22f80db90c4e8ae6e3aeedab6acdb981230).

21. Multi-Agent Designs v6–v9 (2026): AGENTENDESIGN-V6.md to V9.md & IMPLEMENTIERUNG-V7.md, V9.md (experiments/testguided-study/).
22. v6 Pilot Analyses (2026): PILOTANALYSE-V6-20261112-002038-CODEX.md & PILOTANALYSE-20261112-002038.md.
23. v9 Pilot Analysis (2026): PILOTANALYSE-V9-20261112-035522.md (Antigravity / Gemini-3.8-Flash).
24. Multi-Agent Synthesis v6–v9 (2026): ANALYSE-GENERATIONEN-V6-V9.md (Antigravity / Gemini-3.8-Flash, Nov 12, 2026).
25. Submission Audit (2026): KRITISCHE-ANALYSE-PAPER-SUBMISSION.md (Antigravity / Gemini-3.8-Flash, Nov 12, 2026).
26. Revision-1 Grounding Audit (2026): Gutachten & Grounding zur Revision -1 & Developer-Agent-Paper.md Rubric (Antigravity, Nov 12, 2026).